

Homework 5 Solutions

- 问答部分 (29 分)
 - 1 前向传播和反向传播例子 (15 分)
 - 1.1 问题1
 - 1.2 问题2
 - 2 实证问题 (14分)
 - 2.1 问题1: 隐藏单元
 - 2.2 问题2: 学习率
 - 2.3 问题3: 权重初始化
- 编程作业(71分)
 - 3 任务
 - 4 数据集
 - 5 模型定义
 - 5.1 初始化
 - 6 实现
 - 6.1 命令行参数
 - 6.2 输出: 标签文件
 - 6.3 输出度量
 - 7 实现细节
 - 7.1 基于模块的实现方法
 - 7.1.1 模块定义
 - 7.1.2 Module-based AD for Neural Network
 - 7.2 训练过程
 - 7.3 Test Time Procedure
 - 8 提交格式

问答部分 (29 分)

1 前向传播和反向传播例子 (15 分)

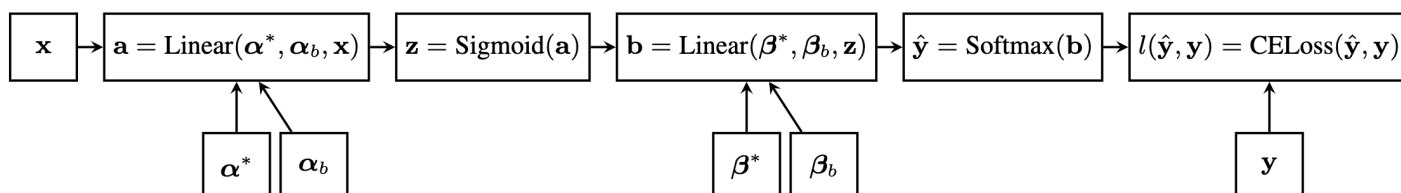


图1:一个隐藏层神经网络的计算图

网络概述：考虑图1中显示的具有一个隐藏层的神经网络。输入层由6个特征 $\mathbf{x} = [x_1, \dots, x_6]^T$ 组成，隐藏层有4个节点 $\mathbf{z} = [z_1, \dots, z_4]^T$ ，输出层是一个概率分布 $\mathbf{y} = [y_1, y_2, y_3]^T$ ，覆盖3个类别（从1开始索引，使得 y_i 是标签 i 的概率）。

α^* 是从输入到隐藏层的权重矩阵， β^* 是从隐藏层到输出层的权重矩阵。

$\alpha_{j,i}^*$ 表示从输入层中的节点 x_i 到隐藏层中的节点 z_j 的权重（例如， $\alpha_{1,2}$ 是从 x_2 到 z_1 的权重）， β^* 的定义类似。我们将使用sigmoid激活函数作为隐藏层，使用softmax作为输出层。

偏置向量 α_b, β_b 定义为： α_b 的第 j 个值（我们记为 $\alpha_{j,b}$ ）是 a_j 的偏置值， β_b 的第 k 个值是 b_k 的偏置值。

网络细节：等效地，我们定义如下的每一项。

输入：

$$\mathbf{x} = [x_1, x_2, x_3, x_4, x_5, x_6]^T \quad (1)$$

第一层(隐藏层)线性组合：

$$a_j = \alpha_{j,b} + \sum_{i=1}^6 \alpha_{j,i}^* \cdot x_i, \forall j \in \{1, \dots, 4\} \quad (2)$$

第一层(隐藏层)激活：

$$z_j = \sigma(a_j) = \frac{1}{1 + \exp(-a_j)}, \forall j \in \{1, \dots, 4\} \quad (3)$$

等效地，我们可以将其写为向量操作，其中将sigmoid激活函数分别应用于向量 $\mathbf{a}$ 的每个元素：

$$\mathbf{z} = \sigma(\mathbf{a}) \quad (4)$$

第二层(输出)线性组合：

$$b_k = \beta_{k,b} + \sum_{j=1}^4 \beta_{k,j}^* \cdot z_j, \forall k \in \{1, \dots, 3\} \quad (5)$$

第二层(输出)激活：

$$\hat{y}_k = \frac{\exp(b_k)}{\sum_{l=1}^3 \exp(b_l)}, \forall k \in \{1, \dots, 3\} \quad (6)$$

损失：我们将使用交叉熵损失， $\ell(\hat{\mathbf{y}}, \mathbf{y})$ 。如果 $\mathbf{y}$ 代表我们的目标输出，这将是一个表示正确类别的one-hot 向量， $\hat{\mathbf{y}}$ 代表网络的输出，那么损失计算如下：

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^3 y_i \log(\hat{y}_i) \quad (7)$$

对于以下问题，请在等式中使用自然对数。

预测：在进行预测时，我们将预测输出层的argmax。例如，如果 $\hat{y}_1 = 0.3, \hat{y}_2 = 0.2, \hat{y}_3 = 0.5$ ，我们将预测类别3。如果训练数据中的真实类别是2，那么我们将得到一个one-hot向量 $\mathbf{y}$ ，其值为 $y_1 = 0, y_2 = 1, y_3 = 0$ 。

1.1 问题1

1. 在以下问题中，你将推导出定义我们神经网络的前面方程的矩阵和向量形式。这是你应该希望编程实现的，以便让你的程序在Gradescope超时限制内运行。

在解决这些问题时，记录向量和矩阵的维度非常重要，以便你能轻松地识别什么是有效的乘法以及什么是无效的乘法。假设你得到了一个训练样例： $\mathbf{x}^{(1)} = [x_1, x_2, x_3, x_4, x_5, x_6]^T$ ，标签类别为**2**，所以 $\mathbf{y}^{(1)} = [0, 1, 0]^T$ 。我们初始化网络权重为：

$$\boldsymbol{\alpha}^* = \begin{bmatrix} \alpha_{1,1} & \alpha_{1,2} & \alpha_{1,3} & \alpha_{1,4} & \alpha_{1,5} & \alpha_{1,6} \\ \alpha_{2,1} & \alpha_{2,2} & \alpha_{2,3} & \alpha_{2,4} & \alpha_{2,5} & \alpha_{2,6} \\ \alpha_{3,1} & \alpha_{3,2} & \alpha_{3,3} & \alpha_{3,4} & \alpha_{3,5} & \alpha_{3,6} \\ \alpha_{4,1} & \alpha_{4,2} & \alpha_{4,3} & \alpha_{4,4} & \alpha_{4,5} & \alpha_{4,6} \end{bmatrix}$$

$$\boldsymbol{\beta}^* = \begin{bmatrix} \beta_{1,1} & \beta_{1,2} & \beta_{1,3} & \beta_{1,4} \\ \beta_{2,1} & \beta_{2,2} & \beta_{2,3} & \beta_{2,4} \\ \beta_{3,1} & \beta_{3,2} & \beta_{3,3} & \beta_{3,4} \end{bmatrix}$$

我们还要考虑偏置项和偏置项的权重 ($\alpha_{j,b}$ 和 $\beta_{k,b}$)。为了考虑这些，我们可以将它们作为一个新列添加到我们初始权重矩阵的开头，以表示偏置（例如， $\alpha_{1,0} = \alpha_{1,b}$ ）。

$$\boldsymbol{\alpha} = \begin{bmatrix} \alpha_{1,0} & \alpha_{1,1} & \alpha_{1,2} & \alpha_{1,3} & \alpha_{1,4} & \alpha_{1,5} & \alpha_{1,6} \\ \alpha_{2,0} & \alpha_{2,1} & \alpha_{2,2} & \alpha_{2,3} & \alpha_{2,4} & \alpha_{2,5} & \alpha_{2,6} \\ \alpha_{3,0} & \alpha_{3,1} & \alpha_{3,2} & \alpha_{3,3} & \alpha_{3,4} & \alpha_{3,5} & \alpha_{3,6} \\ \alpha_{4,0} & \alpha_{4,1} & \alpha_{4,2} & \alpha_{4,3} & \alpha_{4,4} & \alpha_{4,5} & \alpha_{4,6} \end{bmatrix}$$

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_{1,0} & \beta_{1,1} & \beta_{1,2} & \beta_{1,3} & \beta_{1,4} \\ \beta_{2,0} & \beta_{2,1} & \beta_{2,2} & \beta_{2,3} & \beta_{2,4} \\ \beta_{3,0} & \beta_{3,1} & \beta_{3,2} & \beta_{3,3} & \beta_{3,4} \end{bmatrix}$$

然后，我们在输入向量中添加一个对应的新的第一维度，总是设置为1($x_0^{(i)} = 1$)，所以我们的输入变

为：

$$\mathbf{x}^{(1)} = [1, x_1, x_2, x_3, x_4, x_5, x_6]^T$$

(a) (1分) 通过检查初始权重矩阵的形状，我们可以知道神经网络的第一隐藏层有多少个神经元？在你的计数中不要包含偏置。

答：

第一个隐藏层的线性组合可写作如下矩阵乘法的形式：

$$\mathbf{a} = \alpha^* \mathbf{x} + \alpha_b$$

其中， $\mathbf{a} \in \mathbb{R}^{4 \times 1}$ ， $\alpha^* \in \mathbb{R}^{4 \times 6}$ ， $\mathbf{x} \in \mathbb{R}^{6 \times 1}$ ， $\alpha_b \in \mathbb{R}^{4 \times 1}$ 。

隐藏层的神经元数量可以通过查看初始权重矩阵 α^* 的行数来确定。从题干中的矩阵我们可以看到， α^* 有 4 行，不包括偏置。所以答案是4个神经元。

(b) (1分) 我们的神经网络将有多少个输出神经元？

答：

第二个隐藏层的线性组合可写作如下矩阵乘法的形式：

$$\mathbf{b} = \beta^* \mathbf{z} + \beta_b$$

其中， $\mathbf{b} \in \mathbb{R}^{3 \times 1}$ ， $\beta^* \in \mathbb{R}^{3 \times 4}$ ， $\mathbf{z} \in \mathbb{R}^{4 \times 1}$ ， $\beta_b \in \mathbb{R}^{3 \times 1}$ 。

输出层的神经元数量可以通过查看初始权重矩阵 β^* 的行数来确定。从题干中的矩阵我们可以看到， β^* 有 3 行，这意味着输出层有 3 个神经元。所以答案是3个神经元。

(c) (1分) 向量 $\mathbf{a}$ 的元素是由等式2中的 a_j 项组成的（用 $x_i^{(1)}$ 代替 x_i ）。请用 α 和 $\mathbf{x}^{(1)}$ 表示你的答案。

答：

为了将等式(2)转化为矩阵乘法的形式为：

$$\mathbf{a} = \alpha \mathbf{x}^{(1)}$$

其中 $\mathbf{a} \in \mathbb{R}^{4 \times 1}$ ， $\alpha \in \mathbb{R}^{4 \times 7}$ ， $\mathbf{x}^{(1)} \in \mathbb{R}^{7 \times 1}$ 。

(d) (1分) 选择一个：我们不能对我们的权重 β 和向量 $\mathbf{z} = [z_1, z_2, z_3, z_4]^T$ 进行矩阵乘法，因为它们的形状不兼容。以下哪个选项能让我们对 β 和 $\mathbf{z}$ 进行矩阵乘法，使得向量 $\mathbf{b} = \beta \mathbf{z}$ 的条目等价于等式5中的 b_k 值？

- A) 删除 $\mathbf{z}$ 的第一行
- B) 在 $\mathbf{z}$ 的第一项添加一个值1

- C) 在 β 的第一列添加一个额外的列1
- D) 在 β 的第一行添加一行1

答：

已知，权重 $\beta \in \mathbb{R}^{3 \times 5}$ ，而 $\mathbf{z} \in \mathbb{R}^{4 \times 1}$ 。如此显然两者之间不匹配，故在 $\mathbf{z}$ 的开头填加一个1，使其成为 $\mathbf{z}^{(1)} \in \mathbb{R}^{5 \times 1}$ ：

$$\mathbf{z}^{(1)} = [1 \quad z_1 \quad z_2 \quad z_3 \quad z_4]^T$$

故答案选择B

(e) (1分) 输出向量 $\hat{\mathbf{y}}$ 的条目是什么？你的答案应该用 b_1, b_2, b_3 表示。

答：

根据等式(6)，代入可知：

$$\hat{y}_1 = \frac{\exp(b_1)}{\exp(b_1) + \exp(b_2) + \exp(b_3)}$$

$$\hat{y}_2 = \frac{\exp(b_2)}{\exp(b_1) + \exp(b_2) + \exp(b_3)}$$

$$\hat{y}_3 = \frac{\exp(b_3)}{\exp(b_1) + \exp(b_2) + \exp(b_3)}$$

1.2 问题2

2. 我们现在将推导出反向传播算法的矩阵和向量形式，例如

$$\frac{\partial \ell}{\partial \alpha} = \begin{bmatrix} \frac{\partial \ell}{\partial \alpha_{10}} & \frac{\partial \ell}{\partial \alpha_{11}} & \cdots & \frac{\partial \ell}{\partial \alpha_{16}} \\ \frac{\partial \ell}{\partial \alpha_{20}} & \frac{\partial \ell}{\partial \alpha_{21}} & \cdots & \frac{\partial \ell}{\partial \alpha_{26}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial \alpha_{40}} & \frac{\partial \ell}{\partial \alpha_{41}} & \cdots & \frac{\partial \ell}{\partial \alpha_{46}} \end{bmatrix}$$

这一部分你需要推导的数学公式难度大幅提升，你应该始终检查矩阵和向量的形状，并确保你正在将你的矩阵元素与单个导数的计算进行比较，以确保它们匹配（例如，矩阵 $\left(\frac{\partial \ell}{\partial \alpha}\right)_{2,1}$ 的元素应等于 $\frac{\partial \ell}{\partial \alpha_{2,1}}$ ）。记住， ℓ 是我们在等式7中定义的损失函数：

注意：所有向量都是列向量（即，一个 n 维向量 $\mathbf{v} \in \mathbb{R}^{n \times 1}$ ）。假设所有输入到线性层的向量都已经包含了偏置项，除非另有说明。所有偏导数应写成分母布局表示法。分母表示法的一个例子是 $\frac{\partial \ell}{\partial \beta} \in \mathbb{R}^{3 \times 5}$ ，因为 $\beta \in \mathbb{R}^{3 \times 5}$ 。

(a) (1分) 导数 $\frac{\partial \ell}{\partial \hat{y}_i}$ 是多少，其中 $1 \leq i \leq 3$? 你的答案应该用 y_i 和 $\hat{y}_i$ 表示。记住，我们将损失 $\ell(\hat{y}, y)$ 定义如下（注意： $\log$ 是自然对数）：

$$\ell(\hat{y}, y) = - \sum_{i=1}^3 y_i \log(\hat{y}_i) \quad (7)$$

答：

为了找到 $\frac{\partial \ell}{\partial \hat{y}_i}$ ，我们可以直接对等式(7)求导。

对于其他 $j \neq i$ 的项，因为它们相对于 $\hat{y}_i$ 是常数，故导数为0。

我们只考虑含有 i 这一项，其导数为：

$$\frac{\partial}{\partial \hat{y}_i} (-y_i \log(\hat{y}_i)) = -y_i \frac{\partial}{\partial \hat{y}_i} \log(\hat{y}_i) = -y_i \left(\frac{1}{\hat{y}_i} \right)$$

所以，对于每个 $1 \leq i \leq 3$ ，我们都有：

$$\frac{\partial \ell}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$$

(b) (3分) softmax函数关于 b_k 的导数如下：

$$\frac{\partial \hat{y}_l}{\partial b_k} = \hat{y}_l (\mathbb{I}[k = l] - \hat{y}_k) \quad (8)$$

其中， $\mathbb{I}[k = l]$ 是一个指示函数，如果 $k = l$ ，则返回值为1，否则为0。

使用这个和你在(a)中的结果，用灵活的方式写出导数 $\frac{\partial \ell}{\partial b_k}$ ，确保你在等式8中不需要使用指示函数。用 $\hat{y}_k, y_k$ 表示你的解。在下面展示你的工作过程。

提示1： 回忆 $\frac{\partial \ell}{\partial b_k} = \sum_l \frac{\partial \ell}{\partial \hat{y}_l} \frac{\partial \hat{y}_l}{\partial b_k}$ 。

提示2： 在代入你对 $\frac{\partial \ell}{\partial \hat{y}_l}$ 和 $\frac{\partial \hat{y}_l}{\partial b_k}$ 的表达式后，尝试重新排列项，以便其中有表达式 $\hat{y}_k \sum_l y_l$ 的形式。 $\sum_l y_l$ 的值是多少？

答：

首先，我们使用链式法则可知：

$$\frac{\partial \ell}{\partial b_k} = \sum_l \frac{\partial \ell}{\partial \hat{y}_l} \frac{\partial \hat{y}_l}{\partial b_k}$$

代入问题1.2(a)中结果和公式(8)，可得：

$$\begin{aligned}\frac{\partial \ell}{\partial b_k} &= \sum_l \left[-\frac{y_l}{\hat{y}_l} \hat{y}_l (\mathbb{I}[k=l] - \hat{y}_k) \right] \\ &= -\sum_l y_l (\mathbb{I}[k=l] - \hat{y}_k)\end{aligned}$$

这里可以进一步拆分为两部分，前面一半有指示函数，故仅当 $l = k$ 时才不为 0。我们可以这样来表示：

$$\frac{\partial \ell}{\partial b_k} = -y_k + \sum_l y_l \hat{y}_k$$

将 $\hat{y}_k$ 提出来后，简化可得：

$$\frac{\partial \ell}{\partial b_k} = -y_k + \hat{y}_k \sum_l y_l$$

因为 y_l 是一个one-hot编码向量，故 $\sum_l y_l = 1$ 。

所以最终有：

$$\frac{\partial \ell}{\partial b_k} = -y_k + \hat{y}_k$$

(c) (2分) 导数 $\frac{\partial \ell}{\partial \beta}$ 是多少？你的答案应该用 $\frac{\partial \ell}{\partial \mathbf{b}}$ 和 $\mathbf{z}$ 表示。你应该首先考虑这个矩阵中的一个元素：

$$\frac{\partial \ell}{\partial \beta_{kj}}.$$

答：

首先，在问题1.1(d)中，我们得到了：

$$\mathbf{b} = \beta \mathbf{z}$$

注意在本小问中 $\mathbf{z} = [1 \quad z_1 \quad z_2 \quad z_3 \quad z_4]^T$

根据链式法则，对于矩阵中的每一个元素 β_{kj} 有：

$$\frac{\partial \ell}{\partial \beta_{kj}} = \frac{\partial \ell}{\partial b_k} \cdot \frac{\partial b_k}{\partial \beta_{kj}}$$

而又由：

$$b_k = \sum_{j=1}^5 \beta_{kj} z_j$$

可得：

$$\frac{\partial b_k}{\partial \beta_{kj}} = z_j$$

代入上式中，可得：

$$\frac{\partial \ell}{\partial \beta_{kj}} = \frac{\partial \ell}{\partial b_k} \cdot z_j$$

为了表示整个矩阵 $\frac{\partial \ell}{\partial \beta}$ ，我们可以使用外积形式：

$$\frac{\partial \ell}{\partial \beta} = \frac{\partial \ell}{\partial \mathbf{b}} \mathbf{z}^T$$

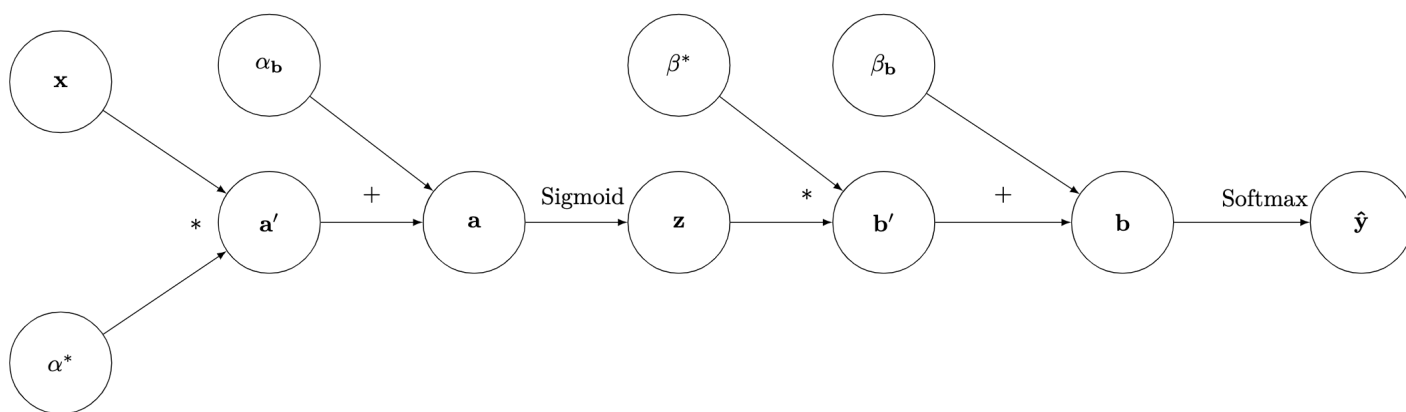
其中 $\frac{\partial \ell}{\partial \mathbf{b}} \in \mathbb{R}^{3 \times 1}$ ， $\mathbf{z} \in \mathbb{R}^{5 \times 1}$ ，故两矩阵相乘可得 $\frac{\partial \ell}{\partial \beta} \in \mathbb{R}^{3 \times 5}$ ，与 β 的维度相同。

(d) (1分) 选择一项：为什么我们在计算导数矩阵 $\frac{\partial \ell}{\partial \alpha}$ 时，使用矩阵 β^* （矩阵 β 去掉第一列的偏置值）而不是 β ？（提示：试着画一个把偏置项展开的计算图）。

- A) 偏置项不更新，所以没有必要在反向传播中包含它们。
- B) 它是可以保证维度匹配的计算代价最小的列。
- C) 元素 $\beta_{k,0}$ 不由 α 的值决定
- D) 对偏置项的损失导数总是零。

答：

如图所示的 $\mathbf{a}'$ 和 $\mathbf{b}'$ 是仍未偏置的矩阵，画出计算图如下：



答案是C

因为 $\beta_{k,0}$ 就是图中的 β_b 这个偏置值，它是一个常数，并且不受隐藏层前的权重 α 的任何影响。因此，当我们计算关于 α 的损失导数时，我们不需要考虑这个偏置项。

(e) (1分) 导数 $\frac{\partial \ell}{\partial z}$ （不包括偏置项）是多少？你的答案应该用 $\frac{\partial \ell}{\partial \mathbf{b}}$ 和 β^* 表示。

答：

本小问中的 $\mathbf{z}$ 首位应该没有加入 1，因为本问中需要的是导数，没有必要对一个常数 1 求导数。

如问题1.2(d)中计算图所示， $\mathbf{b} = \mathbf{b}' + \beta_b$ ，故 $\frac{\partial \mathbf{b}}{\partial \mathbf{b}'} = 1$ 。使用链式法则展开后可得：

$$\frac{\partial \ell}{\partial \mathbf{z}} = \frac{\partial \ell}{\partial \mathbf{b}} \frac{\partial \mathbf{b}}{\partial \mathbf{b}'} \frac{\partial \mathbf{b}'}{\partial \mathbf{z}} = \frac{\partial \ell}{\partial \mathbf{b}} \frac{\partial \mathbf{b}'}{\partial \mathbf{z}}$$

我们知道：

$$\mathbf{b}' = \boldsymbol{\beta}^* \mathbf{z}$$

考虑到 $\mathbf{b}'$ 是关于 $\mathbf{z}$ 的线性函数，我们有：

$$\frac{\partial \mathbf{b}'}{\partial \mathbf{z}} = \boldsymbol{\beta}^*$$

注意，我们还不能直接让矩阵相乘，还需要先考察其各自的维度。其中 $\boldsymbol{\beta}^* \in \mathbb{R}^{3 \times 4}$ ， $\frac{\partial \ell}{\partial \mathbf{b}} \in \mathbb{R}^{3 \times 1}$ 。

这里的乘积是矩阵与向量的乘积，所以我们需要确保维度是正确的。我们因此需要进行矩阵转置以确保维度正确：

$$\frac{\partial \ell}{\partial \mathbf{z}} = (\boldsymbol{\beta}^*)^T \frac{\partial \ell}{\partial \mathbf{b}}$$

其中 $\frac{\partial \ell}{\partial \mathbf{z}} \in \mathbb{R}^{4 \times 1}$ 与 $\mathbf{z}$ 的维度相同。

注意：在对矩阵使用链式法则时，可以与一般变量一样来求。在获得最终结果中有哪些矩阵后，就需要根据各矩阵的维度不同，通过调换位置或是转置等方法，“凑”出正确答案形式。

这种方法虽然在数学上不太严谨，但实际结果全部是正确的。

(f) (1分) 导数 $\frac{\partial \ell}{\partial a_j}$ 用 $\frac{\partial \ell}{\partial z_j}$ 和 z_j 表示是多少？

答：

首先，根据公式(3)，可知第一层激活层为sigmoid函数：

$$z_j = \sigma(a_j) = \frac{1}{1 + \exp(-a_j)}$$

对其求导得：

$$\frac{dz_j}{da_j} = \sigma(a_j)(1 - \sigma(a_j)) = z_j(1 - z_j)$$

使用链式法则，我们可以得到：

$$\frac{\partial \ell}{\partial a_j} = \frac{\partial \ell}{\partial z_j} \cdot \frac{\partial z_j}{\partial a_j}$$

代入上面的结果：

$$\frac{\partial \ell}{\partial a_j} = \frac{\partial \ell}{\partial z_j} \cdot z_j(1 - z_j)$$

(g) (1分) 矩阵 $\frac{\partial \ell}{\partial \alpha}$ 是多少？你的答案应该用 $\frac{\partial \ell}{\partial \mathbf{a}}$ 和 $\mathbf{x}^{(1)}$ 表示。

答：

首先根据问题1.1(c)，我们已知：

$$\mathbf{a} = \alpha \mathbf{x}^{(1)}$$

为了求 $\frac{\partial \ell}{\partial \alpha}$ ，我们可以先求出其中的每一个元素 $\frac{\partial \ell}{\partial \alpha_{ij}}$ ，由链式法则可得：

$$\frac{\partial \ell}{\partial \alpha_{ij}} = \frac{\partial \ell}{\partial a_j} \cdot \frac{\partial a_j}{\partial \alpha_{ij}} = \frac{\partial \ell}{\partial a_j} \cdot x_i^{(1)}$$

其中， $\frac{\partial a_j}{\partial \alpha_{ij}} = x_i^{(1)}$ 非常显然。

接下来，就是将单独的元素，整合成矩阵的形式：

$$\frac{\partial \ell}{\partial \alpha} = \frac{\partial \ell}{\partial \mathbf{a}} \mathbf{x}^{(1)T}$$

其中 $\frac{\partial \ell}{\partial \alpha} \in \mathbb{R}^{4 \times 7}$ ， $\frac{\partial \ell}{\partial \mathbf{a}} \in \mathbb{R}^{4 \times 1}$ ， $\mathbf{x}^{(1)} \in \mathbb{R}^{7 \times 1}$ 。

2 实证问题（14分）

以下问题应在你完成这次作业的编程部分后完成。对于任何绘图问题，你必须使用曲线/线图，为你的图形命名，标记你的坐标轴并提供单位（如适用），并提供图例，以便获得满分。

对于这些问题，使用小数据集。除非另有说明，否则使用以下值作为超参数：

Parameter	Value
Number of Hidden Units	50
Weight Initialization	RANDOM
Learning Rate	0.001

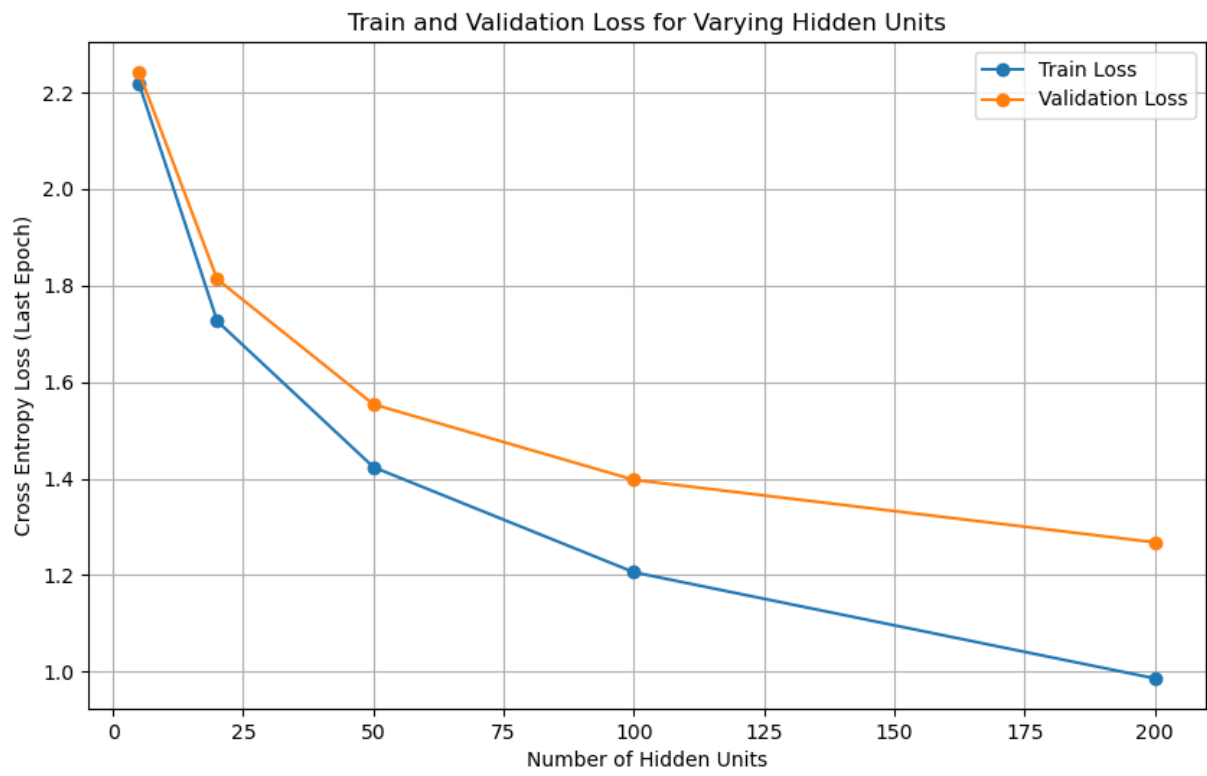
请为所有部分提交计算机生成的图表。为了获得满分，你的图表必须是带有标签的线图，两个轴都有标签，标签上标注了该轴上的值，每条线都有标签。

2.1 问题1：隐藏单元

(a) (2分) 使用上表中提到的超参数训练一个单隐藏层神经网络，除了隐藏单元数应在5, 20, 50, 100, 和 200之间变动。每次运行优化100个epoch。

在y轴上绘制最后一个epoch的平均训练交叉熵（训练数据集上的交叉熵项之和除以训练样本总数），在x轴上绘制隐藏单元数。在同一图中，绘制平均验证交叉熵。x轴应为隐藏单元数，y轴应为平均交叉熵损失，应有一条曲线代表验证损失，一条曲线代表训练损失。

答：



(b) (2分) 检查并评论训练和验证交叉熵的图表。隐藏单元过少会出现什么问题，为什么会这样？

答：

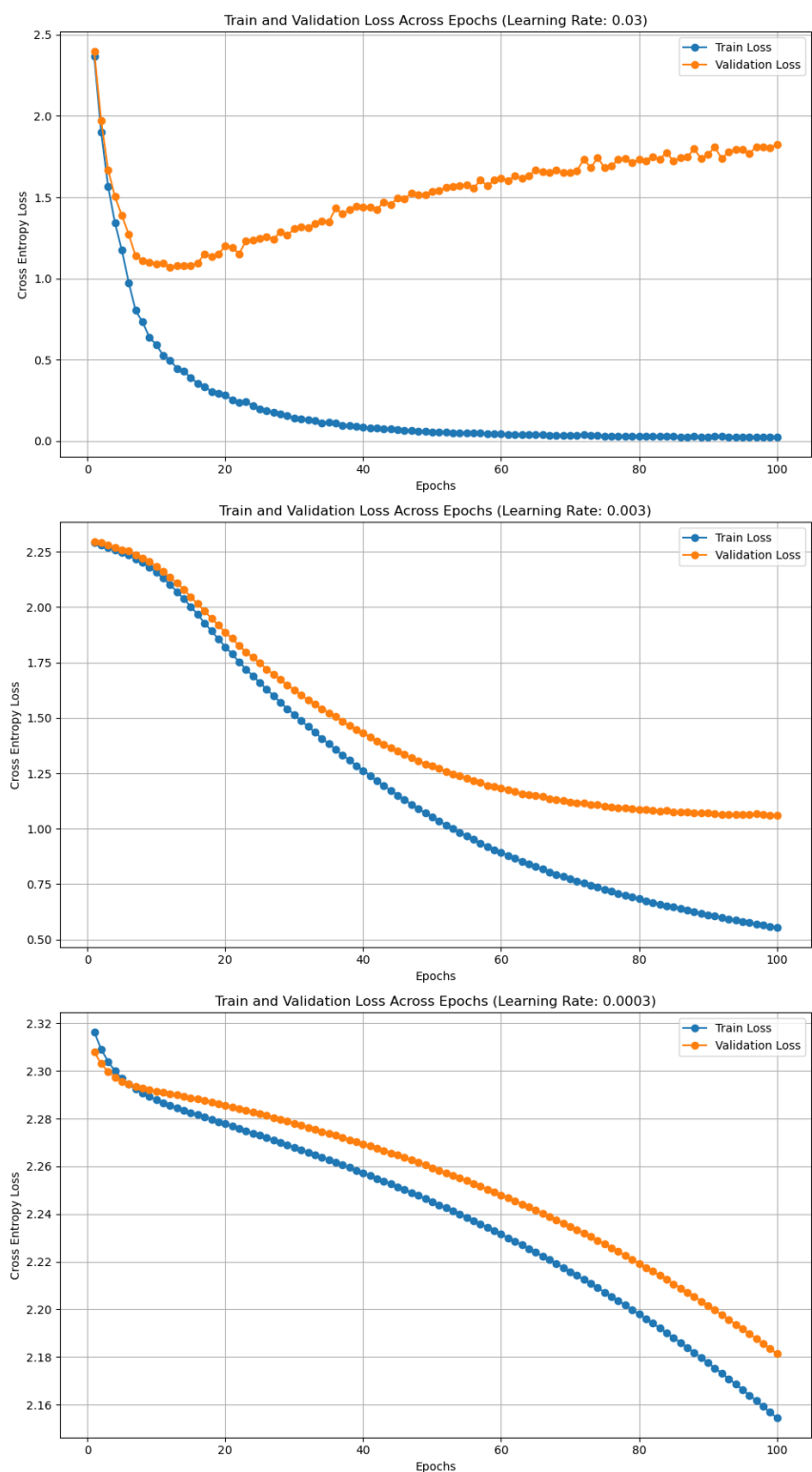
1. 当隐藏单元数较少时，可能导致欠拟合，即模型太简单，无法捕获数据的真实结构。这在图中可能表现为较高的训练和验证损失。
2. 隐藏单元数量增加，则模型的容量也增加，因此，我们可以看到训练损失随着隐藏单元数的增加而减少。
3. 但是，当隐藏单元数量过多时（例如，200），虽然训练损失可能会进一步减少，但验证损失可能不会有太大改善。这是因为模型可能开始过拟合。

2.2 问题2：学习率

(a) (6分) 使用上表中提到的超参数训练一个单隐藏层神经网络，除了学习率应在0.03, 0.003, 0.0003之间变动。每次运行优化100个epoch。

在y轴上绘制平均训练交叉熵，x轴上绘制epoch数，对于提到的学习率。在同一图中，绘制平均验证交叉熵损失。对于每个学习率，绘制一个单独的图。x轴应为epoch数，y轴应为平均交叉熵损失，应有一条曲线代表训练损失，一条曲线代表验证损失。

答：



(b) (2分) 检查并评论训练和验证交叉熵的图表。是否有任何学习率没有达到收敛？是否有任何学习率展现出其他问题？如果有，请描述这些问题，列出引起这些问题的学习率。

答：

学习率为**0.03**时，训练损失快速下降，但验证损失开始上升，此时产生了过拟合。

学习率为**0.003**时，训练和验证损失都随着epoch数的增加而持续下降，并且最后开始平稳，本图中已经达到了收敛。

学习率为**0.0003**时，损失下降得非常缓慢，那么学习率可能太小了，根本就没有收敛。

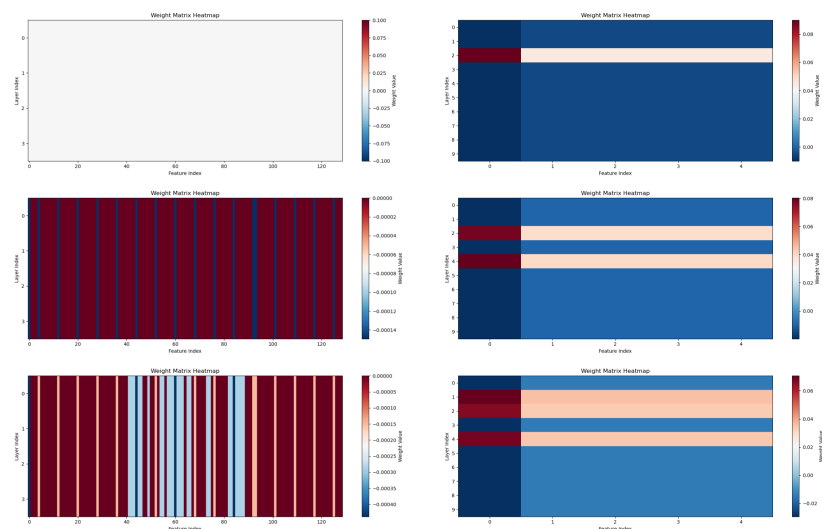
2.3 问题3：权重初始化

(a) (2分) 对于这个练习，你可以在任何数据集上操作。初始化 α 和 β 为零，并在前几次更新后打印它们。例如，你可以使用以下命令开始：

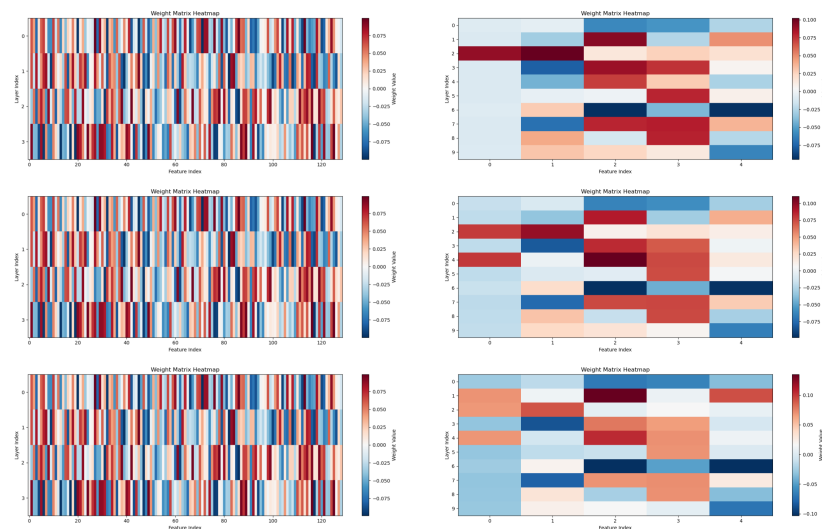
```
$ python neuralnet.py small_train.csv small_validation.csv \  
small_train_out.labels small_validation_out.labels \  
small_metrics_out.txt 1 4 2 0.1 --debug True
```

比较 α 和 β 的行和列中的值。描述观察到的行为以及这可能如何影响模型的容量和收敛。

答：



初始化为0时的权重Heatmap



随机初始化时的权重Heatmap

从上面的结果中，我们可以观察到以下的现象：

初始化为0时，每个神经元经过多次迭代，依然都有相同的输出。因为每一个神经元的输出都相同，那么其梯度更新也都相同。也就是说，无论网络有多少个隐藏单元，都只会执行相同的操作，这就降低了模型的容量，使其无法捕捉到数据中的复杂模式。而随机初始化时，则会打破对称性，使得每个神经元可以开始学习不同的特征。

而且对比可以看出来，初始化为0时，收敛速度非常慢。而随机初始化时，收敛速度大大加快。

编程作业(71分)

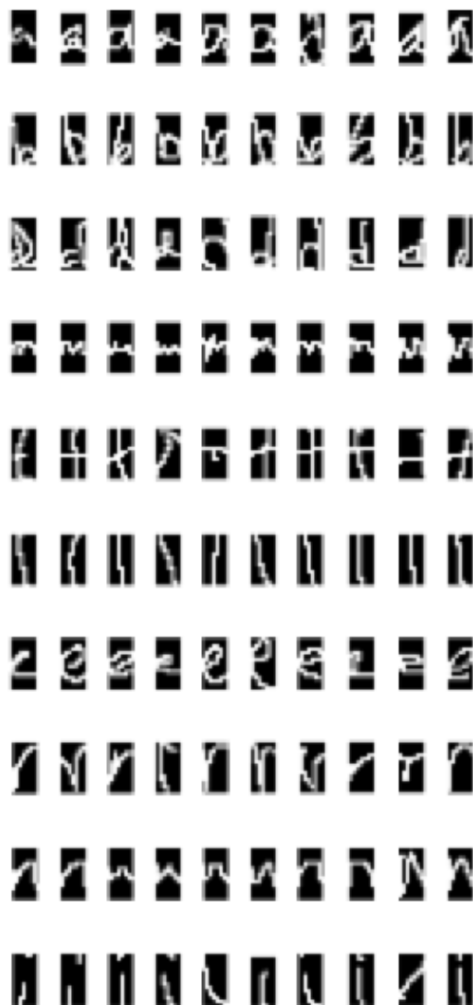


图2:OCR数据集中10个字母的10个随机图像。

3 任务

你在这个作业中的目标是实现一个用于分类图像的单隐藏层神经网络。

4 数据集

我们将使用光学字符识别（OCR）数据集的一个子集。这些数据包括所有26个手写字母的图像；我们的子集将只包括字母"a", "e", "g", "i", "l", "n", "o", "r", "t" 和 "u"。教程提供了一个小型数据集，每个类别有60个样本（50个用于训练，10个用于验证）。我们还会在一个中等大小的数据集上评估你的代码，每个类别有500个样本（400个用于训练，100个用于验证）。图2显示了从数据集中随机抽取的几个字母的10个图像的样本（不是我们在这个作业中要分类的同样的字母）。

文件格式

每个数据集（小型、中型和大型）都由两个csv文件组成——训练和验证。每一行包含由逗号分隔的129列。第一列包含标签，第2列到第129列代表 16×8 图像的像素值，以行优先格式排列。标签0对应"a"，1对应"e"，2对应"g"，3对应"i"，4对应"l"，5对应"n"，6对应"o"，7对应"r"，8对应"t"，9对应"u"。

因为原始图像是黑白的（不是灰度的），所以像素值要么是0要么是1。然而，你应该编写你的代码以接受范围在 $[0, 1]$ 之间的任意像素值。图2中的图像是通过将这些像素值转换成.png文件进行可视化产生的。注意这里没有进行特征工程；相反，你构建的神经网络将学习适合字符识别任务的特征。

5 模型定义

在这个作业中，你将实现一个单隐藏层神经网络，隐藏层使用sigmoid激活函数，输出层上使用softmax函数。假设输入向量 $\mathbf{x}$ 的长度为 M ，隐藏层 $\mathbf{z}$ 由 D 个隐藏单元组成。此外，让输出层 $\hat{\mathbf{y}}$ 是 K 个类别上的概率分布。也就是说，输出向量的每个元素 $\hat{y}_k$ 代表 $\mathbf{x}$ 属于类别 k 的概率。

我们可以通过假设 $x_0 = 1$ 是输入上的偏置特征， $z_0 = 1$ 也是固定的，来紧凑地表示这个模型。这样，我们就有两个参数矩阵 $\boldsymbol{\alpha} \in \mathbb{R}^{D \times (M+1)}$ 和 $\boldsymbol{\beta} \in \mathbb{R}^{K \times (D+1)}$ 。每个矩阵的额外第0列（即 $\boldsymbol{\alpha}_{:,0}$ 和 $\boldsymbol{\beta}_{:,0}$ ）包含偏置参数。

$$\begin{aligned}a_j &= \sum_{m=0}^M \alpha_{j,m} x_m \\z_j &= \sigma(a_j) = \frac{1}{1 + \exp(-a_j)} \\b_k &= \sum_{j=0}^D \beta_{k,j} z_j \\\hat{y}_k &= \text{Softmax}(\mathbf{b}) = \frac{\exp(b_k)}{\sum_{l=1}^K \exp(b_l)}\end{aligned}$$

我们将使用的训练神经网络的目标函数是训练数据集 $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}$ 上的平均交叉熵：

$$J(\alpha, \beta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (9)$$

在等式9中， J 是模型参数 α 和 β 的函数，因为 $\hat{y}_k^{(i)}$ 是神经网络应用于 $\mathbf{x}^{(i)}$ 的输出，因此间接地是 $\mathbf{x}^{(i)}$ ， α 和 β 的函数。 $\hat{y}_k^{(i)}$ 和 $y_k^{(i)}$ 分别是 $\hat{\mathbf{y}}^{(i)}$ 和 $\mathbf{y}^{(i)}$ 的第 k 个组件。

为了训练，你应该使用随机梯度下降（SGD）优化这个目标函数，其中每个训练样本的参数梯度通过反向传播计算。你应该在执行SGD时使用提供的shuffle函数对训练点进行洗牌，将epoch号作为随机种子传入。注意，SGD对目标函数有轻微的影响，因为我們是在"求和"当前点 i ，而不是整个数据集：

$$J_{SGD}(\alpha, \beta) = -\sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (10)$$

你将使用(希望在这一点上)熟悉的SGD更新规则来更新模型的参数：

$$\alpha_{t+1} \leftarrow \alpha_t - \gamma \frac{\partial J_{SGD}(\alpha_t, \beta_t)}{\partial \alpha_t} \quad (11)$$

$$\beta_{t+1} \leftarrow \beta_t - \gamma \frac{\partial J_{SGD}(\alpha_t, \beta_t)}{\partial \beta_t} \quad (12)$$

其中， γ 是学习率， α_t 和 β_t 是步骤 t 时 α 和 β 的值（ α_{t+1} 和 β_{t+1} 同理）。

5.1 初始化

为了使用深度网络，我们首先必须初始化网络中的权重和偏置。这通常是通过随机初始化，或者从一些其他训练过程中初始化权重。对于这个任务，我们将使用两种可能的初始化：

RANDOM: 权重从-0.1到0.1的均匀分布中随机初始化。偏置参数初始化为零。

ZERO: 所有权重初始化为0。

你的代码必须要支持这两种初始化方案。

6 实现

编写一个程序 `neuralnet.py`，使用一个隐藏层的神经网络实现光学字符识别，激活函数为sigmoid。你的程序应该在训练数据上学习模型的参数，报告每个epoch结束时的交叉熵在训练和验证数据上的值，并在训练结束时写出它在两个数据集上的预测和错误率。

你的实现必须满足以下要求：

- 隐藏层使用**sigmoid**激活函数，输出层使用**softmax**，确保形成正确的概率分布。

- 隐藏层的**隐藏单元数**应由命令行标志确定。（关于命令行标志的更多细节在下面提供。）
- 支持两种不同的**初始化策略**，如5.1节所述，通过命令行标志在它们之间选择。
- 使用随机梯度下降（SGD）来优化一个隐藏层神经网络的参数。**epoch**数将作为命令行标志指定。
- 通过命令行标志设置**学习率**。
- 在洗牌后的训练数据上进行随机梯度下降更新。对于每个epoch，你必须重新洗牌**原始**文件数据，而不是前一个epoch的数据。
- 你可以假设输入数据总是有相同的输出标签空间（即 $0, 1, \dots, 9$ ）。除此之外，不要将数据集的任何方面硬编码到你的代码中。我们将在包括不同示例的多个数据集上对你的程序进行自动评分。
- 如果在输出层 $\hat{y}$ 中存在平局，预测最小的索引为标签。（提示：如果你使用正确的NumPy函数，你不需要为平局写额外的代码。）
- 不要使用任何机器学习库。你只可以使用NumPy实现。

实现神经网络可能会有些棘手：参数不仅仅是一个简单的向量，而是许多参数的集合；模型本身的计算效率变得至关重要；初始化策略对整体学习质量有巨大的影响；我们不会改变的其他方面（比如激活函数，优化方法）也有很大的影响。以下提示应该能帮助你：

- 尽可能"向量化"你的代码——对于Python来说这尤其重要。例如，在Python中，你想要避免for循环，而是依赖numpy调用来执行如矩阵乘法、转置、减法等操作，一次性地在整个numpy数组上进行。为什么？因为那些调用可能会更快！这些操作实际上是用快速的C代码实现的，不会像高级脚本语言Python那样变慢。
- 实现一个有限差分测试来检查你的反向传播实现是否正确计算了梯度。如果你选择这样做，在你的反向传播开始给出正确的结果并在提交到Gradescope之前注释掉此功能，因为它会减慢你的代码速度。

6.1 命令行参数

autograder运行并评估生成的文件的输出，使用以下命令：

```
$ python3 neuralnet.py [args...]
```

上面的[args...]是九个命令行参数的占位符：<train_input> <validation_input> <train_out> <validation_out> <metrics_out> <num_epoch> <hidden_units> <init_flag> <learning_rate>。这些参数在下面详细描述：

1. <train_input>：训练输入.csv文件的路径（参见第4节）
2. <validation_input>：验证输入.csv文件的路径（参见第4节）
3. <train_out>：预测训练数据应写入的输出.labels文件的路径（参见第6.2节）
4. <validation_out>：预测验证数据应写入的输出.labels文件的路径（参见第6.2节）
5. <metrics_out>：输出.txt文件的路径，应将训练和验证错误等度量写入该文件（参见第6.3节）
6. <num_epoch>：指定反向传播循环遍历所有训练数据次数的整数（例如，如果 <num_epoch> 等于 5，那么每个训练样本将在反向传播中使用5次）。

7. `<hidden_units>` : 指定隐藏单元数的正整数。
8. `<init_flag>` : 取值为1或2的整数, 指定是否使用RANDOM或ZERO初始化 (参见第5.1节和第5节)。也就是说, 如果 `init_flag==1` , 那么你的权重将从 $[-0.1, 0.1]$ 的均匀分布中随机初始化 (即RANDOM) , 如果 `init_flag==2` , 所有权重都初始化为零 (即ZERO) 。对于这两种设置, **总是将偏置项初始化为零**。
9. `<learning_rate>` : 指定SGD学习率的浮点值。
10. `<--debug>` : (可选参数) 设置日志级别, 设置为DEBUG以显示日志。

例如, 如果你用Python实现了你的程序, 以下命令行将用4个隐藏单元在手册中提供的小数据上运行你的程序, 使用零初始化和0.1的学习率, 运行2个epoch。

```
python3 neuralnet.py small_train.csv small_validation.csv \
small_train_out.labels small_validation_out.labels \
small_metrics_out.txt 2 4 2 0.1
```

6.2 输出：标签文件

你的程序应该写两个输出 `.labels` 文件, 包含你的模型在训练数据(`<train_out>`)和验证数据(`<validation_out>`)上的预测。每个文件应包含每个实例的预测标签, 每个标签打印在新的一行。使用 `\n` 来创建一个新的行。

你的标签应该与参考实现完全匹配, 这将通过运行你的程序并将你的输出文件与参考解决方案进行比较来由自动评分程序检查。

注意: 你应该使用原始训练数据中的相同整数标识符输出你的预测标签。你还应该在每个序列的末尾插入一个空行 (使用 `\n`) , 就像在输入数据文件中那样。

6.3 输出度量

生成一个文件, 在其中报告以下度量:

交叉熵: 在每个epoch之后, 报告训练数据上的平均交叉熵 `crossentropy (train)` 和验证数据上的 `crossentropy (validation)` (参见公式9) 。这两个交叉熵值应在每个epoch结束时报告, 并以epoch号为前缀。例如, 在通过训练示例的第二次之后, 这些应以epoch=2为前缀。你打印出来的总训练损失数应等于num_epoch, 验证损失同理。

错误: 在最后一个epoch之后 (即, 当训练完全完成时) , 报告最终的训练错误 `error (train)` 和验证错误 `error (validation)` 。

下面给出了一个样本输出。它包含了使用上述命令时, 前2个epoch的训练和验证损失, 以及最终的错误率。

```
epoch=1 crossentropy(train): 2.1415670910950144
epoch=1 crossentropy(validation): 2.1502231738985618
epoch=2 crossentropy(train): 1.8642629963917074
epoch=2 crossentropy(validation): 1.8780601379038728
error(train): 0.73
error(validation): 0.72
```

请确保你的输出格式与上面显示的完全相同。词"epoch"和epoch数字之间有一个等号=，但没有空格。在epoch数字后应有一个空格（例如，epoch=1后面的空格），在度量值前的冒号后也应有一个空格（例如，epoch=1 crossentropy (train) :后面的空格）。每一行应以Unix行结束符\n结束。

```
python3 neuralnet.py tiny_train.csv tiny_validation.csv \
tiny_train_out.labels tiny_validation_out.labels \
tiny_metrics_out.txt 1 4 2 0.1
```

特定的输出文件名并不重要，但请确保保持其他参数与上面显示的完全一样。

7 实现细节

7.1 基于模块的实现方法

基于模块的自动微分（AD）是一种长期用于开发深度学习库的技术，也是你在这个作业中被鼓励遵循的实现方法。动态神经网络包是那些允许在运行时动态指定计算图的包，如Torch¹，PyTorch²和DyNet³，这些都使用了我们在这里将要描述的基于模块的AD⁴。

¹ <http://torch.ch/>

² <http://pytorch.org/>

³ <https://dynet.readthedocs.io>

⁴ 静态神经网络包是那些需要静态指定计算图的包，随后这些计算图会被编译成代码。例如Theano、Tensorflow 和 CNTK 就是这类库的例子。这些库也是基于模块的，但其实现形式与我们在此推荐的动态方法不同。

基于模块的AD的关键思想是将神经网络的计算组件化到层。每一层可以被认为将计算图中的众多节点（其中的一个子集）整合到一个向量值的节点中。这样的向量值节点应能做到以下几点，我们称每一个为一个模块（对应于Python中的一个类）：

1. 给定输入 $\mathbf{a} = [a_1, \dots, a_A]$ ，通过某个可微函数 f ，计算输出 $\mathbf{b} = [b_1, \dots, b_B]$ 。也就是说， $\mathbf{b} = f(\mathbf{a})$ 。
2. 给定输出的梯度 $\mathbf{g}_\mathbf{b} = \frac{\partial J}{\partial \mathbf{b}} = \left[\frac{dJ}{db_1}, \dots, \frac{dJ}{db_B} \right]$ ，计算输入的梯度 $\mathbf{g}_\mathbf{a} = \frac{\partial J}{\partial \mathbf{a}} = \left[\frac{dJ}{da_1}, \dots, \frac{dJ}{da_A} \right]$ ，其中 J 是整个计算图的最终实值输出。这是通过链式法则 $\frac{dJ}{da_i} = \sum_{j=1}^B \frac{dJ}{db_j} \frac{\partial b_j}{\partial a_i}$ 对所有 $i \in \{1, \dots, A\}$ 完成的。

7.1.1 模块定义

在我们的实现中，我们为神经网络定义的模块对应于一个线性层和一个Sigmoid层。虽然我们可以额外为Softmax和Cross-Entropy定义模块，但为了简便，我们将它们作为函数（尽管如果你愿意，你也可以将它们变为模块）。每个模块都定义了一个前向方法 $\mathbf{b} = * \cdot \text{FORWARD}(\mathbf{a})$ ，和一个反向方法 $\mathbf{g}_a = *. \text{BACKWARD}(\mathbf{g}_b)$ 。换句话说，前向方法给定输入 a ，产生输出 $\mathbf{b}$ ；同时，反向方法给定输出的梯度 $\mathbf{g}_b$ ，产生输入的梯度 $\mathbf{g}_a$ 。每个模块也可能存储适当的值（例如，线性层存储权重矩阵 α, β ）。

注意，对于线性模块，虽然输入和输出的梯度被传入和传出模块，但权重矩阵的梯度 $\mathbf{g}_\alpha$ 和 $\mathbf{g}_\beta$ 并没有。这遵循了面向对象设计原则的封装—— $\mathbf{g}_\alpha$ 和 $\mathbf{g}_\beta$ 只被它们各自的线性层所需要，所以我们只在线性模块本身中存储它们。后来，它们将被用于一个SGD更新，这将由一个额外的STEP方法执行。（或者，由于这个作业的SGD更新总是每个示例应用一次，你可以直接在BACKWARD中执行SGD更新，尽管你应该对你的操作的顺序特别小心。）

此外，如果你已经完成了笔试问题2，你可能会注意到，尽管我们只将 $\mathbf{g}_b$ ，即模块输出的梯度，传入 $*. \text{BACKWARD}(\mathbf{g}_b)$ ，但我们可能需要更多的东西来计算某些层的梯度。具体来说，如果你检查你的梯度表达式，你可能会注意到它们使用了前向传播的一些值。因此，在你的前向方法中，你会想要缓存一些值，在后向传播中稍后使用。在初始代码中，我们通过作为类属性的缓存字典来实现这一点，其中你可以存储参数名称作为 keys，映射到它们的缓存值。

最后，你需要密切关注你传入和从模块返回的维度。维度 A 和 B 特定于模块，我们有输入 $\mathbf{a} \in \mathbb{R}^A$ ，输出 $\mathbf{b} \in \mathbb{R}^B$ ，输出的梯度 $\mathbf{g}_a \triangleq \nabla_{\mathbf{a}} J \in \mathbb{R}^A$ ，以及输入的梯度 $\mathbf{g}_b \triangleq \nabla_{\mathbf{b}} J \in \mathbb{R}^B$ 。

我们为你提供了线性模块的伪代码作为示例。

Linear Module

```
1: procedure FORWARD(a)
2:   Compute b using this layer's weight matrix
3:   Cache intermediate value(s) for the backward pass           ▷ See Written Question 1.2(c)
4:   return b
5: procedure BACKWARD(gb)
6:   Bring the necessary cached values into scope
7:   Compute  $\mathbf{g}_\alpha$ 
8:   Compute  $\mathbf{g}_a$ 
9:   Store  $\mathbf{g}_\alpha$  for subsequent SGD update
10:  return  $\mathbf{g}_a$ 
11: procedure STEP()
12:   Apply SGD update to weights  $\alpha$  using stored gradient  $\mathbf{g}_\alpha$ 
```

7.1.2 Module-based AD for Neural Network

假设我们的一个隐层神经网络是由模块组成的，我们可以使用这些模块定义正向和反向传播的函数，如下所示：

Algorithm 1 Forward Computation

```
1: procedure NNFORWARD(Training example  $(\mathbf{x}, \mathbf{y})$ )
2:    $\mathbf{a} = \text{LINEAR1.FORWARD}(\mathbf{x})$  ▷ First linear layer module
3:    $\mathbf{z} = \text{SIGMOID.FORWARD}(\mathbf{a})$  ▷ Sigmoid activation module
4:    $\mathbf{b} = \text{LINEAR2.FORWARD}(\mathbf{z})$  ▷ Second linear layer module
5:    $\hat{\mathbf{y}} = \text{SOFTMAX}(\mathbf{b})$  ▷ Softmax function
6:    $J = \text{CROSSENTROPY}(\mathbf{y}, \hat{\mathbf{y}})$  ▷ CrossEntropy function
7:   return  $J, \hat{\mathbf{y}}$ 
```

Algorithm 2 Backpropagation

```
1: procedure NNBACKWARD( $\mathbf{y}, \hat{\mathbf{y}}$ )
2:    $g_J = \frac{\partial J}{\partial J} = 1$  ▷ Base case
3:    $\mathbf{g}_b = \text{DSOFTMAXCROSSENTROPY}(\mathbf{y}, \hat{\mathbf{y}}, g_J)$  ▷ See Written Question 1.2(b)
4:    $\mathbf{g}_z = \text{LINEAR2.BACKWARD}(\mathbf{g}_b)$ 
5:    $\mathbf{g}_a = \text{SIGMOID.BACKWARD}(\mathbf{g}_z)$ 
6:    $\mathbf{g}_x = \text{LINEAR1.BACKWARD}(\mathbf{g}_a)$  ▷ We discard  $\mathbf{g}_x$ 
```

这里有一个重要的收获:我们实际上可以将这两个函数视为它们自己定义了另一个模块! 它是一个1隐层神经网络模块。也就是说, 单个训练样本的神经网络的交叉熵本身是一个可微函数, 我们知道如何计算其输入的梯度, 给定其输出的梯度。

7.2 训练过程

考虑在第5节中描述的神经网络应用于第 i 个训练样本 $(\mathbf{x}, \mathbf{y})$, 其中 $\mathbf{y}$ 是真实标签的独热编码。我们的神经网络输出为 $\hat{\mathbf{y}} = h_{\alpha, \beta}(\mathbf{x})$, 其中 α 和 β 分别是第一层和第二层的参数, 而 $h_{\alpha, \beta}$ 是一个具有 Sigmoid 激活函数和 Softmax 输出的单隐藏层神经网络。损失函数是负交叉熵

$J = \ell(\hat{\mathbf{y}}, \mathbf{y}) = -\mathbf{y}^T \log(\hat{\mathbf{y}})$ 。实际上, $J = J_{\mathbf{x}, \mathbf{y}}(\alpha, \beta)$ 是我们的训练样本 $(\mathbf{x}, \mathbf{y})$ 以及模型参数 α, β 的函数, 尽管我们为了简洁只写了 J 。

为了训练我们的神经网络, 我们将使用随机梯度下降 (SGD)。因为我们希望你的程序在 Gradescope 上的测试中表现得大致确定, 我们要求:

- (1) 你应该使用我们提供的 shuffle 函数在每个 epoch 开始时打乱你的数据,
- (2) 你要使用固定的学习率。
- (3) 为了解决问题 2.3, 你应该在 epoch 的前几次更新时, 用 `print_weights()` 函数打印权重。

SGD 的过程如下, 其中 E 是 epoch 的数量, γ 是学习率。

Algorithm 3 Training with Stochastic Gradient Descent (SGD)

```
1: procedure SGD(Training data  $\mathcal{D}_{train}$ , test data  $\mathcal{D}_t$ )
2:   Initialize parameters  $\alpha, \beta$  ▷ Use either RANDOM or ZERO from Section 5.1
3:   for  $e \in \{1, 2, \dots, E\}$  do ▷ For each epoch
4:      $\mathcal{D} = \text{SHUFFLE}(\mathcal{D}_{train}, e)$ 
5:     for  $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$  do ▷ For each training example
6:       Compute neural network forward prop:
7:        $J, \hat{\mathbf{y}} = \text{NN.FORWARD}(\mathbf{x}, \mathbf{y}, \alpha, \beta)$ 
8:       Compute gradients via backprop:
9:       
$$\left. \begin{aligned} \mathbf{g}_\alpha &= \frac{\partial J}{\partial \alpha} \\ \mathbf{g}_\beta &= \frac{\partial J}{\partial \beta} \end{aligned} \right\} \text{ given by NN.BACKWARD}(\mathbf{y}, \hat{\mathbf{y}})$$

10:      Update parameters with SGD updates  $\mathbf{g}_\alpha, \mathbf{g}_\beta$ :
11:       $\alpha \leftarrow \alpha - \gamma \mathbf{g}_\alpha$ 
12:       $\beta \leftarrow \beta - \gamma \mathbf{g}_\beta$ 
13:      Evaluate training mean cross-entropy  $J_{\mathcal{D}}(\alpha, \beta)$ 
14:      Evaluate test mean cross-entropy  $J_{\mathcal{D}_t}(\alpha, \beta)$ 
15:   return parameters  $\alpha, \beta$ 
```

7.3 Test Time Procedure

在测试时，我们为每个示例输出最可能的预测：

Algorithm 4 Prediction at Test Time

```
1: procedure PREDICT(Unlabeled train or test dataset  $\mathcal{D}'$ )
2:   for  $\mathbf{x} \in \mathcal{D}'$  do
3:     Compute neural network prediction  $\hat{\mathbf{y}} = h(\mathbf{x})$ 
4:     Predict the label with highest probability  $l = \text{argmax}_k \hat{y}_k$ 
```

8 提交格式

1. 将问答部分的答案，整理成PDF格式，命名为 HW5_姓名_学号.pdf 。
2. 编程作业部分，只需提交 neuralnet.py 这一个文件，并将其重命名为 neuralnet_姓名_学号.py 。
3. 将上述两文件打包为不加密的 .zip 格式，并命名为 HW5_姓名_学号.zip 。